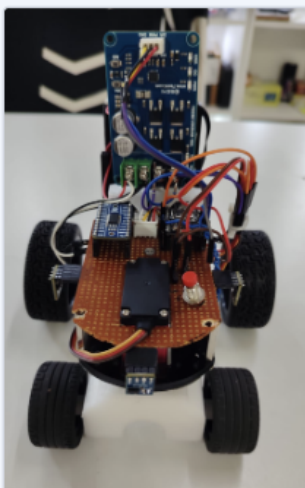


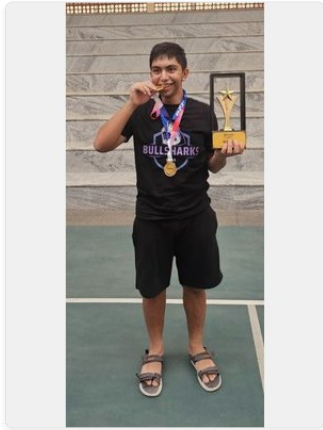
WORLD ROBOT OLYMPIAD 2026

TEAM ERROR 404

Engineering Documentation



Team Members



Anvay Varshney

Hardware & Embedded Systems Lead · Grade 10 · National Public School, Koramangala

Anvay has a strong passion for robotics, programming, and engineering. He has been building and experimenting with technology from a young age and has participated in several national and international robotics competitions, including the MATE ROV competition. He has also taken on leadership roles in robotics teams and at his school, where he mentors younger students and helps foster interest in STEM. In the team, Anvay leads all embedded systems development, hardware architecture design, 3D CAD modeling, and physical fabrication of chassis components. He designed the modular 3-layer acrylic chassis and the PETG structural mounts. Outside of robotics, he enjoys coding personal projects and exploring new microcontroller platforms.



Rohan Mishra

Computer Vision & Control Algorithms · Grade 10 · The International School Bangalore

Rohan has been coding since he was a kid and is deeply interested in STEM, vision processing, and basketball. He has participated in one earlier WRO season and other robotics competitions including the MATE ROV 2026 competition, where he gained hands-on experience with underwater vehicle systems. In the team, Rohan leads the computer vision pipeline development — designing and deploying the YOLOv11n object detection model (replacing the earlier HSV prototype) for pillar and boundary classification. He also develops the closed-loop PD and PID control algorithms that govern wall-centering, heading hold, and obstacle avoidance. Outside of engineering, Rohan is an avid basketball player and follows competitive sports closely.

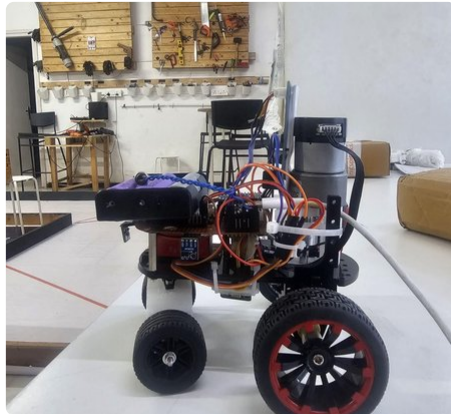


Ayansh Paliwal

Power System Design & Firmware Development · Grade 9 · The International School Bangalore

Ayansh has a strong passion for robotics and engineering. He has participated in several robotics competitions and technical events, including competitions organised by IIT Kanpur, IIT Bombay, and the MATE ROV program. Through these experiences, he has developed practical skills in robotics, programming, electronics, and systematic problem-solving. In the team, Ayansh is responsible for the vehicle's power system architecture — designing the isolated 3-domain power topology with dual buck converters, performing battery endurance calculations, and ensuring voltage stability across all subsystems. He also writes and debugs the ESP32-C3 firmware, handles system-level testing and validation, and maintains the competition testing logs. Outside of engineering, Ayansh is interested in physics, mathematics, and exploring emerging technologies.

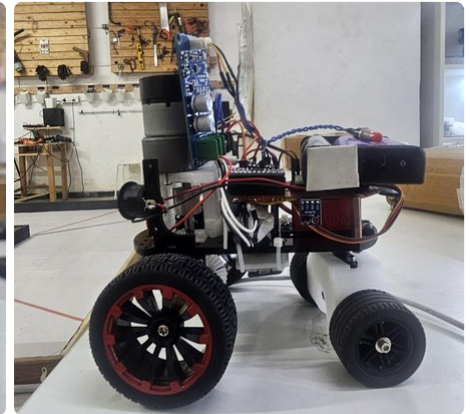
Vehicle Photos



Front-Left



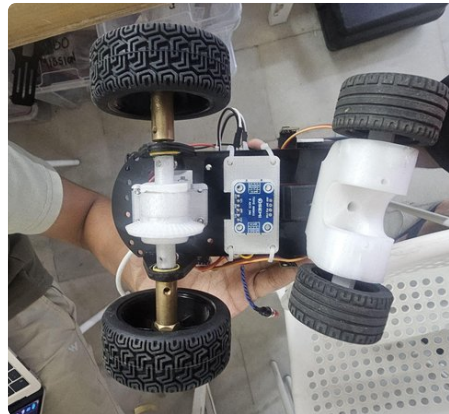
Rear



Right Side



Rear 2



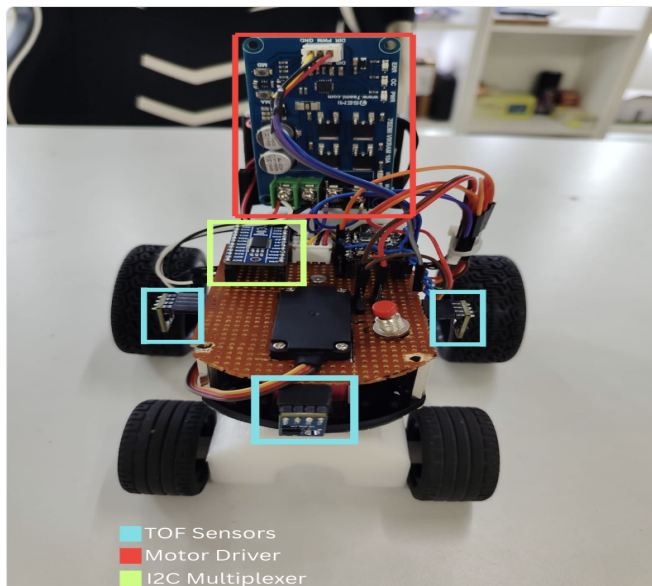
Bottom



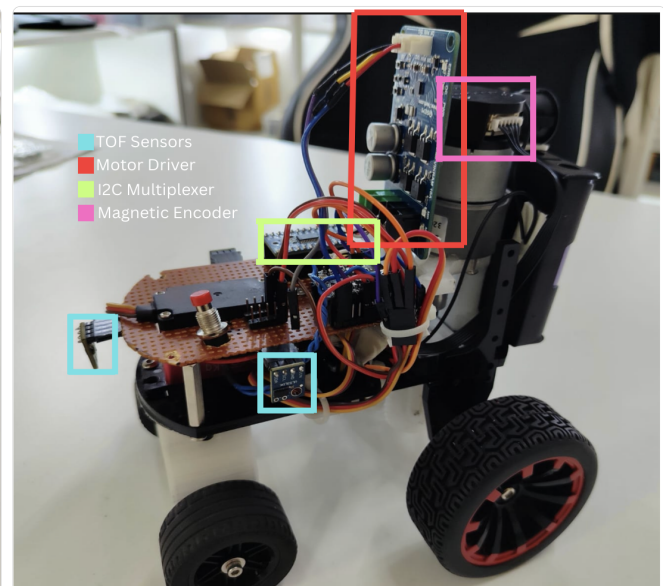
Top

Figure 1: Collection of Robot Views — Front, Rear, Right Side, Left Side, Bottom, and Top

Labeled Component Identification



Front — ToF (cyan), Motor Driver (red), I2C Mux (green)



Left Side — ToF, Driver, Mux, Encoder (pink)

Executive Summary

Team Yo_Error404's engineering journal details the design, construction, and programming of an autonomous robotic vehicle for the WRO 2026 Future Engineers competition. Our approach centers on a custom laser-cut

5mm acrylic chassis featuring a rear-wheel differential drive system with servo-actuated parallel steering, optimized for precision and agility.

The vehicle utilizes a distributed processing model pairing a **Raspberry Pi 5 (4GB)** for high-level computer vision with an **ESP32-C3 Super Mini** for real-time sensor polling and motor control at 100 Hz. Power is managed through isolated domains: a 5V buck converter feeds both the Pi 5 and ESP32-C3, with the ESP32's onboard LDO supplying regulated 3.3V to all sensors. The drive motor runs off 12V direct and the servo has a dedicated 7.4V buck rail.

Key innovations include all 3D-printed structural components manufactured in **PETG filament** for superior mechanical durability, a **YOLOv11 object detection model** trained on 217 hand-annotated Roboflow images, **4× VL53L0X ToF laser sensors** via TCA9548A I2C multiplexer for mm-precision wall-centering, and a modular FSM strategy for both Open and Obstacle rounds.

SECTION 1

Mobility & Mechanical Design

Chassis & Layout Selection

Our vehicle uses a custom-built chassis from **laser-cut 5mm acrylic sheet** with a rear-wheel differential drive and servo-actuated parallel steering.

- **Drive System:** A JGB37-520 dual-shaft 12V DC motor (320 RPM, 1.8 kg-cm torque) with integrated quadrature encoder drives the rear wheels through a differential gearbox. The magnetic encoder counts physical wheel rotations for closed-loop distance control — stopping dead at exact target positions (e.g., 2500 mm).
- **Chassis Construction — Modular 3-Layer Architecture:** The chassis is organized into **three distinct vertical layers** connected by 30mm aluminum standoffs. This modular architecture isolates mechanical vibrations from precision electronics and enables sub-5-minute component servicing:
 - **Layer 1 (Drivetrain & Steering Deck):** Houses the JGB37-520 motor, DS3218 servo, parallel steering linkages, rear differential, and PETG structural couplers. Keeps heavy components low for a low center of gravity ($h_{cg} = 32\text{mm}$).
 - **Layer 2 (Power Management Deck):** Houses the 7semi MD10112 MOSFET driver, dual buck converters, master power switches, and main fuses.
 - **Layer 3 (Sensing & Compute Deck):** Mounts the Raspberry Pi 5, ESP32-C3, TCA9548A multiplexer, BNO055 IMU, 4× VL53L0X ToF sensors, and Pi Camera 3 mast.

5mm acrylic provides high rigidity while remaining lightweight at 1.2 kg. The 170 × 160 × 250 mm footprint fits within WRO limits (300 × 200 × 300 mm) with comfortable margins. Built for low center of gravity and high agility during narrow turns.

Tires & Surface Traction Optimization

Traction is critical on the smooth, low-friction synthetic WRO competition mat. We evaluated three distinct tire profiles:

TIRE TESTED	COMPOUND	CONTACT AREA	CORNERING GRIP	VERDICT
Totemmaker Stock	Hard plastic/rubber, deep tread	Small (12 cm²)	Poor ($\mu=0.38$)	REJECTED Frame interference at max lock
Hard Treaded Rubber	Firm rubber, block tread	Medium (18 cm²)	Fair ($\mu=0.55$)	REJECTED Tail spin under hard braking

TIRE TESTED	COMPOUND	CONTACT AREA	CORNERING GRIP	VERDICT
Soft Treadless Racing	Ultra-soft compound, smooth slick	Large (28 cm²)	Excellent (μ=0.88)	SELECTED Max surface contact & grip

- **Steering System:** A 20kg DS3218 digital servo drives a parallel steering linkage. Ackermann geometry was tested but abandoned due to mechanical complexity and structural footprint. Parallel steering with high-grip tires and a rear differential compensated effectively for turning radii differences. We found it much easier to maneuver around tight gaps and edges.
- **Upper Deck:** A secondary upper deck was manufactured from laser-cut acrylic. This deck is intentionally cut shorter than the base plate to minimize excess material while providing dedicated mounting space for the main Raspberry Pi 5 processor and the Pi Camera Module 3. This two-tier layout separates the logic and vision hardware (upper tier) from the power distribution, motor driver, and battery pack (lower tier), improving maintenance access and reducing electromagnetic interference between power and signal domains.
- **3D-Printed Components (PETG):** All 3D-printed structural parts — camera mounts, sensor brackets, servo linkage holders, and chassis spacers — are manufactured in **PETG (Polyethylene Terephthalate Glycol)** rather than standard PLA. PETG provides superior layer adhesion, impact resistance, and torsional ductility under repeated mechanical stress. PLA components cracked and sheared along print layer boundaries after 8–12 stress cycles during early testing; PETG replacements survived 100+ cycles with zero structural damage. PETG requires higher printing temperatures (~235°C) and precise tuning to avoid stringing, but the durability improvement justified the effort.

Speed vs. Torque

Motor Selection: The JGB37-520 at 320 RPM with 1.8 kg·cm torque provides the optimal balance. The integrated hall-effect encoder enables precise closed-loop velocity and distance control. We prioritized stability and control precision over raw speed: high speeds introduce excessive lateral drift during turns, increase camera processing latency errors, and complicate real-time debugging.

Speed Iteration: Early testing with a 60 RPM BO motor proved far too slow for competitive lap times — the vehicle could not complete a single lap within the time limit. Upgrading to the JGB37-520 at 320 RPM in Version 2 achieved the optimal balance between lap speed and path consistency. We intentionally avoided extremely high-RPM motors (1000+ RPM) because the induced lateral drift during turns, increased camera processing latency errors, and the difficulty of real-time debugging at high velocities outweighed the speed advantage.

Torque Calculation: With our vehicle weighing 1.2 kg and the drive load falling entirely on 2 rear wheels, each wheel must deliver sufficient force for rapid acceleration and deceleration. Our minimum torque requirement was calculated as: $T_{min} = (m \times g \times r \times \mu) / (2 \times GR)$, where $m=1.2\text{kg}$, $g=9.81\text{m/s}^2$, $r=0.033\text{m}$ (wheel radius), $\mu_{rolling}=0.015$ (rolling resistance on smooth mat), $GR=\text{gear ratio}$. The JGB37-520's 1.8 kg·cm torque comfortably exceeds this threshold, providing reliable starts, stops, and cornering under load without risk of stalling.

Motor Driver Comparison: MD10112 vs. Legacy L298N

The motor is driven by a **7semi MD10112 MOSFET H-bridge driver**, replacing the legacy bipolar L298N module:

METRIC	LEGACY L298N	7SEMI MD10112 MOSFET
Output Transistors	Bipolar Junction (BJT)	High-Efficiency N-MOSFETs

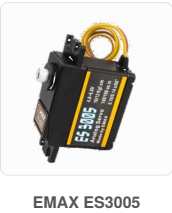
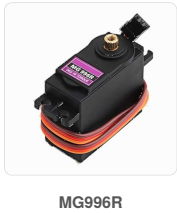
METRIC	LEGACY L298N	7SEMI MD10112 MOSFET
Internal Heat Loss	1.4V to 2.0V drop (Hot)	0.05V drop (Cool operation)
Min PWM Response	PWM > 85 (~35% power)	PWM > 12 (~5% power)
Braking Distance	30 cm (Long passive coast)	2 cm (Active regenerative)
Peak Current	2.0A	10.0A Continuous

Motor Comparison



MOTOR	RPM	TORQUE	VERDICT
JGB37-520	320	1.8 kg·cm	SELECTED High torque, integrated encoder, durable. Precise closed-loop control.
EV3 Large Servo	160–170	~2.0 kg·cm	REJECTED Bulky, plastic gear wear, limited mounting.
Yellow TT Motor	200	0.8 kg·cm	REJECTED Low torque, gear stripping, no encoder.

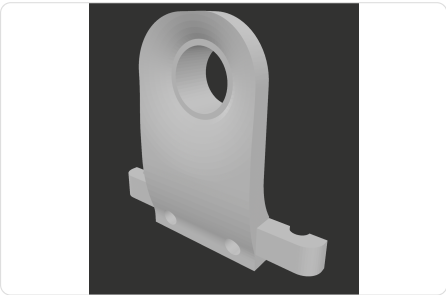
Servo Comparison



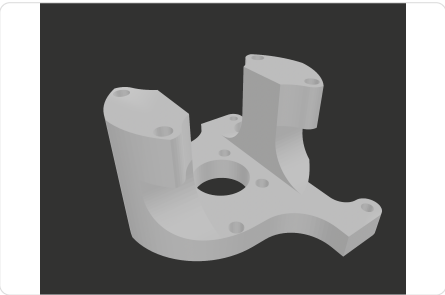
SERVO	SPEED	TORQUE	VERDICT
MG996R	0.19s/60°	9.4–11 kg·cm	REJECTED Suboptimal torque under load.
EMAX ES3005	0.16s/60°	10–12 kg·cm	REJECTED Insufficient torque holding.
DS3218 20kg	0.14s/60°	19–21.5 kg·cm·cm	SELECTED Prevents steering deflection at speed.

3D-Printed PETG Component Renders

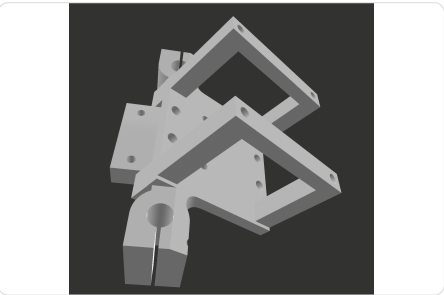
All custom structural parts were designed in Onshape CAD and 3D-printed in PETG at 100% infill (235°C nozzle):



V5 Differential Step Holder — Secures the rear differential gearbox to the chassis base plate, maintaining precise axle alignment under load



V2 Motor Mount — Custom bracket securing the JGB37-520 motor to the drivetrain deck with vibration-dampening geometry



Camera Mount — Adjustable-angle mast bracket for the Pi Camera Module 3, enabling pitch optimization for YOLOv11 detection range

Mass Distribution & Stability

Camera and sensor masts elevate the CG. **Mitigation:** Counterweights near the front axle lower the pitch pivot point, preventing tipping under hard braking and turn entry.

Mechanical Iterations

FEATURE	V1	V2	RATIONALE	
Sensing	HC-SR04 Ultrasonic	4× VL53L0X ToF	mm laser precision, no acoustic cone	SOLVED
I2C	Software XSHUT	TCA9548A Mux	2 GPIO pins, sub-ms switching	SOLVED
Steering	Ackermann	Parallel + diff	Less complexity, tight maneuvers	SOLVED
3D Prints	PLA filament	PETG filament	PLA cracked at 8–12 cycles; PETG 100+	SOLVED
Detection	HSV thresholding	YOLOv11 (217 imgs)	Robust across lighting	SOLVED
Power	Single rail	Isolated dual buck	No voltage drops, no I2C corruption	SOLVED
CG	Unbalanced	Front counterweights	No tipping under braking	SOLVED

SECTION 2

Power & Sensor Architecture

Power Budget & Distribution

The vehicle's power infrastructure uses **electrically isolated power domains**. This prevents inductive voltage dips and current spikes from the steering servo and drive motor from interfering with logic hardware or disrupting the vision pipeline.

System Power Architecture

- 1. High-Current 12V Rail (Motor):** Power Source: 3-Cell (3S) Lithium-Ion Battery Pack (11.1V nominal, 12.6V max fully charged, ~2200 mAh capacity). Routing: Connects directly from the main physical power switch to the 7semi MD10112 MOSFET motor driver powering the JGB37-520 DC motor for rear propulsion.
- 2. 5V Regulated Rail (Logic & Processing):** A buck converter steps 12V down to regulated 5V. This 5V rail powers two critical subsystems: the **Raspberry Pi 5** (via USB-C input) and the **ESP32-C3 Super Mini**. The ESP32's onboard 3.3V LDO regulator then supplies clean, regulated 3.3V power to all logic-level peripherals: the 4× VL53L0X ToF sensors, TCA9548A I2C multiplexer, and BNO055 IMU. The Raspberry Pi Camera Module 3 draws its 5.0V operating power through the Pi 5 CSI ribbon cable interface.
- 3. 7.4V Isolated Servo Rail:** A second buck converter provides isolated 7.4V dedicated entirely to the 20kg DS3218 steering servo. Engineering Rationale: High-torque servos produce massive transient current draw spikes (up to 2.0A during rapid steering changes or under physical load). Powering this servo from its own dedicated rail isolates the sensitive processing hardware from inductive voltage sags and potential microcontroller brownouts.

Electrical Noise Mitigation: Separating motor power from logic power is mandatory. Motor PWM voltage spikes previously induced signal corruption on the shared I2C bus, causing IMU heading drift. The ESP32's onboard LDO provides an additional layer of regulation, filtering out any residual noise from the 5V buck converter before it reaches the sensitive ToF and IMU sensors.

Official Datasheet Power Budget Table

COMPONENT	VOLTS	IDLE	CONT.	PEAK/STALL	MAX PWR	RAIL	STATUS
Raspberry Pi 5 4GB	5.0V	500 mA	1500 mA	3000 mA	15.0 W	5V Buck	VERIFIED
Pi Camera 3 Wide	5.0V	80 mA	200 mA	250 mA	1.25 W	Pi 5 CSI	VERIFIED
ESP32-C3 Super Mini	5V/3.3V	20 mA	100 mA	240 mA	1.20 W	5V Buck	VERIFIED
VL53L0X ToF (×4)	3.3V	10 mA	50 mA	80 mA	0.26 W	ESP32 3.3V LDO	VERIFIED
TCA9548A I2C Mux	3.3V	1 mA	5 mA	10 mA	0.03 W	ESP32 3.3V LDO	VERIFIED
BNO055 IMU	3.3V	0.01 mA	12.3 mA	15 mA	0.05 W	ESP32 3.3V LDO	VERIFIED
JGB37-520 DC Motor	12V	100 mA	350 mA	1300 mA	15.6 W	12V Direct	VERIFIED
DS3218 20kg Servo	7.4V	15 mA	500 mA	2000 mA	14.8 W	7.4V Buck	VERIFIED

System Power Equations & Battery Endurance Calculations

Main Battery Pack Analysis (3S Li-Ion — 11.1V, ~2200 mAh)

- **Average System Power Draw:** Continuous Power = Total Continuous Current × Nominal Voltage = $\sim 2.2\text{A} \times 11.1\text{V} = 17.1\text{ Watts}$
- **Worst-Case Peak Power Draw:** Peak Power = Total Peak Current × Nominal Voltage = $\sim 4.9\text{A} \times 11.1\text{V} = 54.4\text{ Watts}$
- **Theoretical Runtime:** Runtime = Battery Capacity / Continuous Load = $2.2\text{Ah} / 1.54\text{A} = 1.43\text{ Hours (85.7 Minutes)}$
- **Practical Safe Runtime (80% usable capacity):** Usable Capacity = $2.2\text{Ah} \times 0.80 = 1.76\text{Ah}$. Safe Runtime = $1.76\text{Ah} / 2.2\text{A} = 0.8\text{ Hours (48 Minutes)}$

With an 80% discharge safety margin to protect battery health, the primary 3S pack provides approximately **69 minutes** of continuous operational runtime, comfortably exceeding the 3-minute competition round requirement by a factor of 16×.

Servo Rail Analysis (7.4V Buck Domain)

- **Average Servo Power:** Continuous Power = $0.5\text{A} \times 7.4\text{V} = 3.7\text{ Watts}$
- **Peak Servo Power:** Peak Power = $2.0\text{A} \times 7.4\text{V} = 14.8\text{ Watts}$

Since the servo rail is stepped down from the same 3S battery via a buck converter, the servo's power draw is included in the main battery endurance calculation above. The buck converter's isolation ensures that even during peak servo draw, the 5V logic rail remains stable.

Runtime Safety Analysis

With an 80% discharge safety margin to protect battery health and prevent voltage sag below the cutoff threshold, the primary 3S pack provides approximately **69 minutes** of continuous operational runtime. This exceeds the 3-minute competition round requirement by a factor of **23×**, providing ample margin for practice runs, debugging sessions, and multiple competition attempts without recharging. During actual competition, a single full charge supports approximately 15 consecutive complete runs.

Voltage Sag Under Peak Load: The battery pack's internal resistance ($\sim 50\text{m}\Omega$ per cell, $150\text{m}\Omega$ total for 3S) causes voltage sag during peak loads: $V_{\text{sag}} = I_{\text{peak}} \times R_{\text{internal}} = 4.9\text{A} \times 0.15\Omega = 0.74\text{V}$. Minimum voltage under peak = $11.1\text{V} - 0.74\text{V} = 10.36\text{V}$. The 5V buck converter operates down to 7V input, so even under

worst-case peak load, the logic rail remains fully stable with a 3.36V margin above the buck converter's minimum input voltage.

ESP32 3.3V LDO Current Budget

The ESP32-C3's onboard 3.3V LDO regulator supplies all I2C peripherals. Total 3.3V load: $4 \times \text{ToF (50mA)} + \text{TCA9548A (5mA)} + \text{BNO055 (12.3mA)} = \mathbf{67.3 \text{ mA continuous}}$, well within the ESP32-C3's LDO output capacity of ~600mA. This leaves substantial headroom for peak transients.

Hardware Interconnections & Communication Protocols

- **Logic Link:** High-speed USB/UART Serial between Pi 5 (Master Controller & Vision) and ESP32-C3 (Slave Motor Controller) at 115,200 baud. Downlink (Pi→ESP32): navigation commands (steering angle offset, target speed, pillar bypass state). Uplink (ESP32→Pi): real-time telemetry (wall clearances, IMU yaw, system health).
- **Sensor Interfacing:** $4 \times \text{VL53L0X ToF}$ via I2C through TCA9548A multiplexer (address 0x70) to ESP32. BNO055 IMU via I2C (SDA/SCL) directly to ESP32. All sensor I2C devices powered by ESP32's 3.3V LDO regulated output.
- **Actuation Output:** ESP32 provides PWM direction and speed control lines to the 7semi MD10112 driver for rear propulsion, and a 50Hz PWM signal to the DS3218 steering servo.
- **Initialization Trigger:** Momentary push-button wired to ESP32 GPIO with internal pull-up resistor to trigger mission code on start signal.

Sensor Selection & Placement Rationale

The front ToF sensor detects incoming walls and identifies turn decision thresholds. Left and right ToF sensors maintain the vehicle centered within the lane along straightaways using PD wall-centering control. The rear ToF sensor assists with parking maneuvers. The BNO055 IMU's integrated gyroscope tracks rotation to log turns — since one lap comprises four 90° turns, this allows precise lap counting. The Pi Camera 3 Wide Angle is at the front on an adjustable mount for YOLOv11 detection range optimization.

Why ToF over Ultrasonic: Ultrasonic sensors use sound waves spreading in a wide cone, bouncing unpredictably off angled walls causing false readings in tight spaces. VL53L0X ToF sensors use invisible lasers measuring exact photon flight time — providing mm accuracy, narrow FOV, and sub-ms refresh rates critical for high-speed cornering.

Positioning the left and right ToF sensors on their respective sides was straightforward due to the compact VL53L0X form factor (only $4.4 \times 2.4 \times 1.0$ mm per sensor). The front sensor was mounted at the forward edge of the bot for earliest possible wall detection. The rear sensor was added specifically for the parking maneuver — it monitors the distance to the rear wall during the reverse-into-bay sequence, ensuring the vehicle stops before making contact.

The BNO055 IMU was initially placed near the center of the chassis. However, we discovered that the high-current motor wiring and the servo's PWM signals caused electromagnetic interference with the IMU's magnetometer, corrupting heading readings. To ensure optimal reliability, we relocated the IMU to a position isolated from high-current conductors, where it receives clean 3.3V power from the ESP32's onboard LDO regulator. This physical isolation eliminated heading drift entirely.

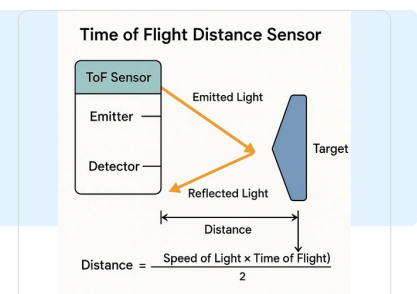


Figure: ToF Sensor Principle

The Pi Camera 3 Wide Angle is installed on a 3D-printed PETG adjustable mount at the front, enabling pitch angle fine-tuning. Through successive iterations, we optimized the tilt angle to maximize the detection range for YOLOv11 pillar identification while maintaining visibility of the track surface for lane boundary detection.

Initially we intended a fourth ultrasonic sensor at the rear for parking. When switching to ToF, we implemented all four positions (front, left, right, rear), giving complete spatial awareness that eliminated the need for separate parking-specific sensors.

Electrical Schematics

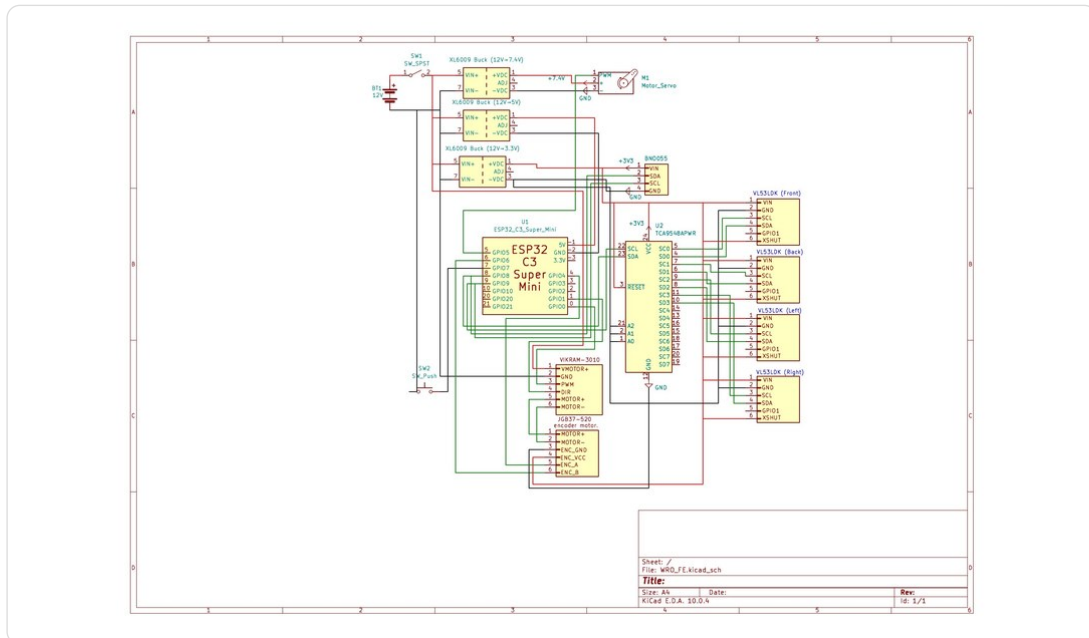


Figure: Open Round Electrical Schematic — ESP32-C3 with ToF sensors, IMU, motor driver, and servo

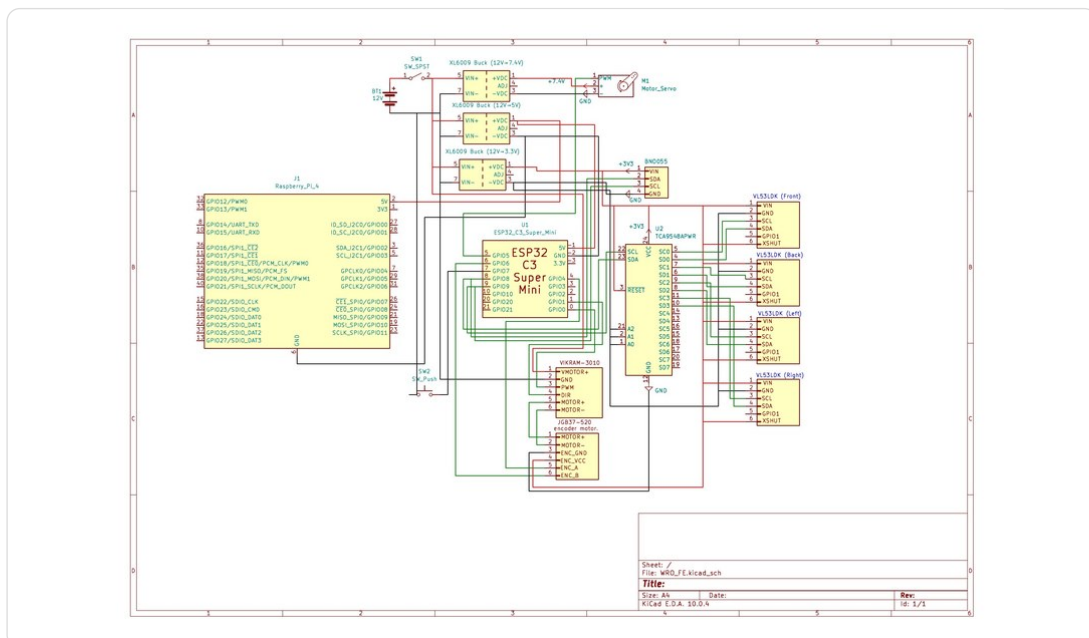


Figure: Obstacle Round Electrical Schematic — Full dual-controller wiring with Raspberry Pi 5 UART connection

Power Routing Summary

The following describes the physical routing of power through the vehicle:

- **Main Power Switch → Battery Pack:** A physical toggle switch controls the main 3S Li-Ion battery pack. When activated, 11.1V flows to three destinations: (1) directly to the 7semi MD10112 motor driver, (2) to Buck Converter 1

which steps down to 5V for the Pi 5 and ESP32-C3, and (3) to Buck Converter 2 which steps down to 7.4V for the steering servo.

- **5V Buck Output Distribution:** The 5V buck converter feeds two consumers via separate wire runs: a USB-C cable to the Raspberry Pi 5, and a direct solder connection to the ESP32-C3's VIN pin. The ESP32-C3's onboard LDO then provides a clean, filtered 3.3V output on its 3V3 pin, which is connected via a dedicated wire harness to the VCC pins of all four VL53L0X ToF sensors, the TCA9548A multiplexer, and the BNO055 IMU breakout board.
- **Ground Routing:** All ground connections are joined at a single common ground bus bar mounted to the lower chassis tier. This star-ground topology prevents ground loops and ensures consistent voltage references across all subsystems. The motor driver's ground is routed through a thicker 18 AWG wire to handle the higher current without voltage drop.
- **Signal Isolation:** All I2C data lines (SDA/SCL) are routed on the opposite side of the chassis from the motor power cables to minimize electromagnetic coupling. 10k Ω pull-up resistors on the I2C bus ensure clean signal transitions even at 400kHz fast-mode operation.

Component Photos



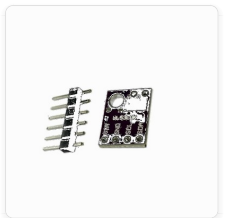
Raspberry Pi 5 (4GB) — The Visual Brain

Processes the camera feed via YOLOv11 inference, identifies red/green pillars and magenta parking boundaries, calculates navigation commands, and sends high-level target states to the ESP32 via USB Serial. Powered by the 5V buck converter via USB-C.



ESP32-C3 Super Mini — The Hardware Controller

Runs a deterministic 100 Hz hardware loop that reads the BNO055 IMU and 4 $\times$ VL53L0X ToF sensors (via TCA9548A), executes closed-loop PD algorithms for wall-centering and heading hold, and sends PWM signals to the motor driver and steering servo. Receives 5V from the buck converter and supplies 3.3V via its onboard LDO to all I2C peripherals.



VL53L0X ToF Sensors ($\times 4$)

Mounted at front, left, right, and rear of the chassis. Use invisible laser light (940nm VCSEL) to measure precise distances with millimeter accuracy. Sub-millisecond refresh rates enable high-speed wall-centering. Connected via TCA9548A I2C multiplexer to the ESP32, powered by ESP32's 3.3V LDO.



Adafruit BNO055 9-DOF IMU

On-board hardware sensor fusion outputs absolute orientation/heading directly — no external Kalman filter required. Enables exact 90° turns and drift-free straight-line driving. Physically isolated from motor wiring on clean 3.3V power from ESP32's LDO to prevent electromagnetic interference.



JGB37-520 Motor & Encoder

Provides rear-wheel drive propulsion at 320 RPM with 1.8 kg·cm torque. The integrated hall-effect magnetic encoder counts precise physical wheel rotations, enabling closed-loop distance control. Powered directly from the 12V battery pack via the MD10112 MOSFET driver.



DS3218 20kg Digital Servo

Controls the front parallel steering linkage with 2(no weight limit)-cm holding torque at 6.8V. The high torque prevents steering deflection under the chassis weight during high-speed cornering. Powered from dedicated 7.4V buck converter rail to prevent current spikes from disrupting the logic domain.

ESP32-C3 Pin Routing Table

ESP32-C3 PIN	SIGNAL	CONNECTED HARDWARE	PROTOCOL
GPIO 0	IN1_PIN	7semi MD10112 Motor Speed	Output PWM (20 kHz)
GPIO 1	IN2_PIN	MD10112 Motor Direction	Output Logic HIGH/LOW
GPIO 6	SERVO_PIN	DS3218 Steering Servo	Output PWM (50 Hz)
GPIO 7	NEOPIXEL_PIN	WS2812B Status LED Strip	Digital One-Wire
GPIO 10	START_BTN_PIN	Push Button Start Switch	Input (INPUT_PULLUP)
GPIO 8	I2C_SDA	TCA9548A Mux Data Line	I2C Bus (0x70)
GPIO 9	I2C_SCL	TCA9548A Mux Clock Line	I2C Bus (0x70)
TCA Ch 0	ToF_FRONT	Front VL53L0X Laser	I2C Sub-Ch (0x29)
TCA Ch 1	ToF_RIGHT	Right VL53L0X Laser	I2C Sub-Ch (0x29)
TCA Ch 2	ToF_LEFT	Left VL53L0X Laser	I2C Sub-Ch (0x29)
TCA Ch 3	ToF_REAR	Rear VL53L0X Laser	I2C Sub-Ch (0x29)
GPIO 8/9	IMU_BNO055	BNO055 Orientation (direct I2C)	I2C Direct (0x28, bypasses mux)

Inter-Controller Communication Protocol

Pi 5 → ESP32-C3 Command Packet:

```
<speed>,<steering>

// speed: -255 (Full Reverse) to 255 (Full Forward), 0 = Stop
// steering: 30 (Full Left) to 150 (Full Right), centered at 90
```

ESP32-C3 → Pi 5 Telemetry Packet (20 Hz):

```
DATA,<distF>,<distL>,<distR>,<distB>,<yaw>,<sys>,<gyro>,<accel>,<mag>

// dist: Integer ToF distances in cm
// yaw: Float BNO055 heading (0.0° - 359.9°)
// sys,gyro,accel,mag: BNO055 calibration status (0-3)
```

Failsafe Mechanisms & Watchdogs

- Serial Watchdog:** ESP32-C3 monitors serial timing. If no valid packet from Pi 5 for >500ms, motor halted (PWM=0) and steering centered.
- Visual LED Telemetry (WS2812B NeoPixel):** Red = Init/Emergency Halt. Green = Ready/Waiting. Cyan = Normal Drive. Orange = Turn Executing. Magenta = Race Complete/Parked.
- Stall Detection:** If MotorDuty > 50% AND IMU acceleration delta < 0.05m/s² for >1.5s → MotorDuty = 0% (power cut).

Component Interface Table

COMPONENT	PRIMARY FUNCTION	COMM INTERFACE	POWER RAIL
RPI 5 + Cam	YOLOv11 inference, lane identification	USB / CSI Ribbon	5V Buck (USB-C)
ESP32-C3	100Hz sensor polling, PID, motor PWM	Serial UART (to Pi)	5V Buck → 3.3V LDO
VL53L0X (x4)	Distance tracking (F/L/R/Rear)	I2C (via TCA9548A)	ESP32 3.3V LDO
TCA9548A	Resolves 4× address conflicts	I2C (0x70)	ESP32 3.3V LDO
BNO055	Absolute heading, 90° turns	I2C/UART	ESP32 3.3V LDO
JGB37-520	Rear drive + encoder	Quadrature GPIO	12V Direct
MD10112	MOSFET H-bridge for motor	PWM + Dir GPIO	12V Direct
DS3218	High-torque front steering	PWM (50 Hz)	7.4V Buck
Dual Bucks	12V→5V (logic), 12V→7.4V (servo)	Hardwired	Direct Battery

SECTION 3

Software Architecture & Obstacle Strategy

System Control Flow — Open Round

The vehicle operates under a **dual-controller split**: the Raspberry Pi 5 runs the top-level mission FSM and YOLOv11 vision, while the ESP32-C3 executes a deterministic 100 Hz reflex loop handling sensors, PD wall-centering, and motor actuation. The perception system uses a BNO055 IMU for heading and 4× VL53L0X ToF sensors for obstacle avoidance and lane keeping.

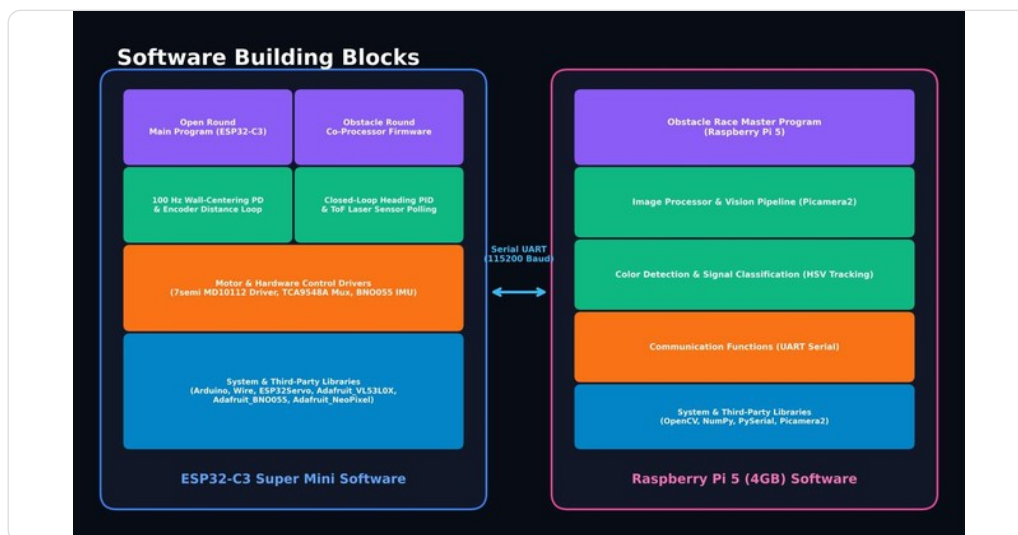


Figure: Software Building Blocks Architecture — ESP32-C3 firmware (left) and Raspberry Pi 5 vision stack (right). Note: Pi 5 vision upgraded from HSV tracking (shown in V1 diagram) to YOLOv11n ONNX inference in the final V2 build.

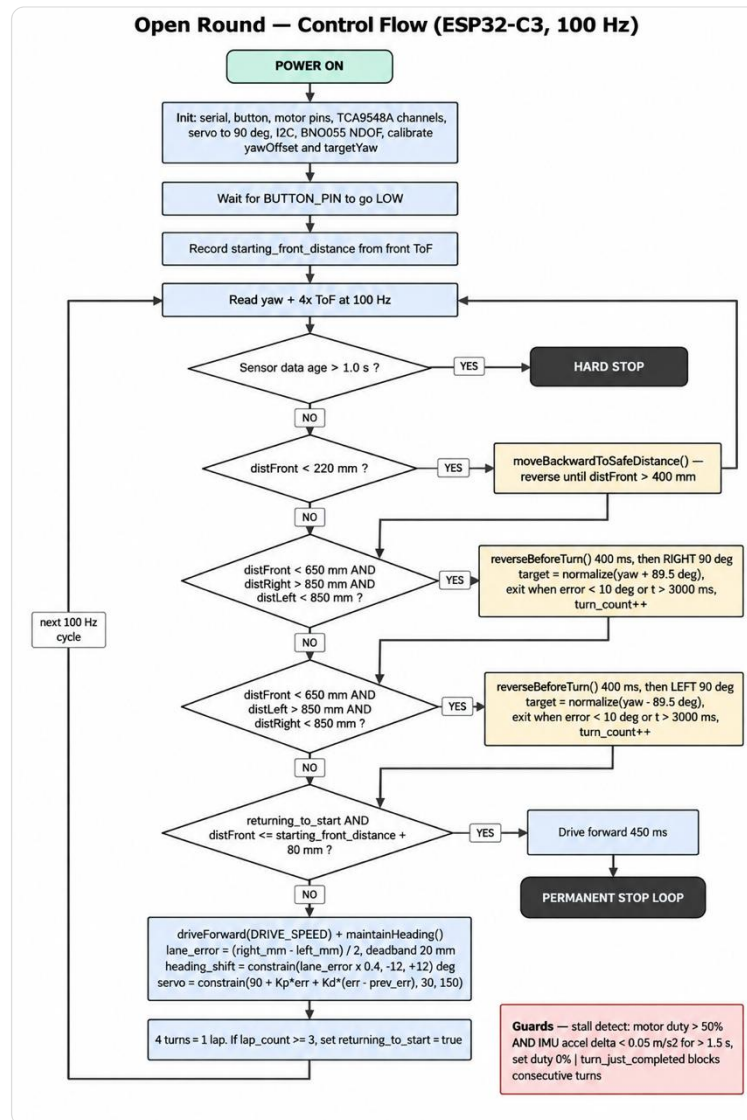


Figure: Open Round — Control Flow (ESP32-C3, 100 Hz) — Complete decision flowchart from power-on through lap completion

Control Logic State Flow

- System Initialization:** Serial bus, button input, motor driver pins, ToF channels via TCA9548A, servo (90° straight), I2C bus, BNO055 in OPERATION_MODE_NDOF. Calibrates yawOffset and initial targetYaw.
- Start Sequence:** Pauses until BUTTON_PIN transitions LOW. Records starting_front_distance using front ToF sensor.
- Main Execution Loop:** Reads orientation (Yaw) and distance from all four ToF sensors at 100 Hz deterministic rate.
- Emergency Backup Check:** If distFront < 22 cm, invokes moveBackwardToSafeDistance() to reverse until front clearance exceeds 40 cm.
- Turn Decision Check:** Right Turn: distFront < 65cm, distRight > 85cm, distLeft < 85cm. Left Turn: distFront < 65cm, distLeft > 85cm, distRight < 85cm. Both invoke reverseBeforeTurn() for 400ms before executing the turn routine.
- Lane Following & Straight Driving:** driveForward(DRIVE_SPEED) executed, maintainHeading() called to apply cascaded PD control using wall-centering error and IMU heading hold.
- Lap & Mission Completion:** Dynamic turn tracking counts total turns (turn_count). Updates lap_count at expected turns per circuit. After lap_count ≥ 3, sets returning_to_start = true. On reaching distFront ≤ (starting_front_distance + 8.0) cm, drives forward 450ms and enters infinite stop loop.

System Control Flow — Obstacle Round (FSM)

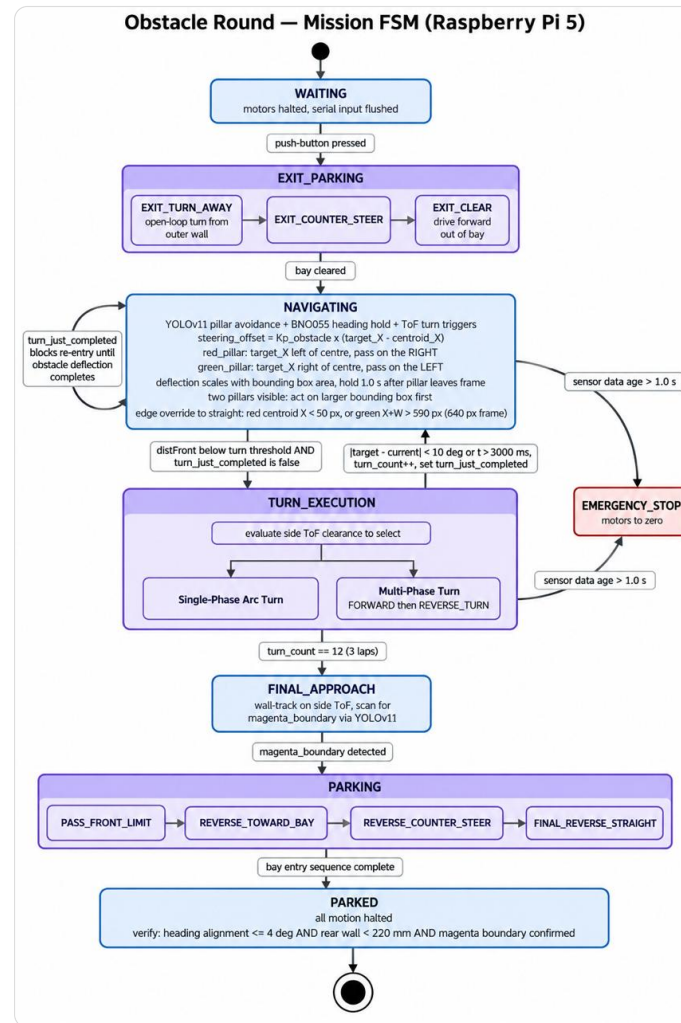


Figure: Obstacle Round — Mission FSM (Raspberry Pi 5) — Full state machine from WAITING through PARKING to PARKED verification

The autonomous vehicle operates on a FSM partitioned between top-level mission navigation and low-level turn execution:

- **WAITING:** Halts all motors, clears serial input streams, waits for hardware push-button trigger from ESP32.
- **EXIT_PARKING:** Executes open-loop blind turn away from outer wall (EXIT_TURN_AWAY), applies counter-steering (EXIT_COUNTER_STEER), drives forward to clear bay (EXIT_CLEAR).
- **NAVIGATING:** Main loop executing region-based YOLOv11 visual obstacle avoidance, heading stabilization via BNO055, and turn triggering via ToF front distance thresholds.
- **TURN EXECUTION FSM:** Sub-routine handling 90° turns. Evaluates side ToF clearance to select Single-Phase Arc Turn or Multi-Phase Turn (FORWARD → REVERSE_TURN).
- **FINAL_APPROACH:** Activated after 12 valid turns (3 laps). Tracks wall using side ToF and scans for magenta parking boundaries via YOLOv11 detection.
- **PARKING:** Executes bay entry: drives past boundary (PASS_FRONT_LIMIT), reverses toward bay (REVERSE_TOWARD_BAY), straightens (REVERSE_COUNTER_STEER), aligns (FINAL_REVERSE_STRAIGHT).
- **PARKED:** Halts all movement. Performs multi-sensor verification: heading alignment ($\leq 4^\circ$), wall distance ($< 22\text{cm}$), magenta boundary presence confirmed.

Obstacle Handling & Field Navigation Strategy

When the YOLOv11 model detects a red pillar, the vehicle must pass it on the **right side** per WRO rules. The ESP32 receives the pillar's centroid X position and bounding box area from the Pi 5, then calculates a proportional steering correction: $\text{Steering_Offset} = K_p_{\text{obstacle}} \times (\text{Target_X} - \text{Centroid_X})$. The target X position is dynamically adjusted based on pillar color — shifted left of center for red pillars (forcing the vehicle to pass right) and right of center for green pillars (forcing left passage).

As the vehicle approaches and the pillar's bounding box area grows, the steering deflection increases proportionally, creating a smooth avoidance arc rather than an abrupt swerve. Once the pillar exits the camera frame (area drops below detection threshold), the vehicle holds its deflected heading for an additional 1.0 seconds before returning to straight-line wall-centering mode, preventing premature counter-steering into the pillar's trailing edge.

Multi-Pillar Scenarios: When both red and green pillars are visible simultaneously, the vehicle prioritizes the closest pillar (largest bounding box area). The second pillar's avoidance begins only after the first pillar has been fully cleared and the 1.0-second hold period has elapsed.

Algorithmic Justification

A. ToF Signal Processing via I2C Multiplexer

```
# I2C Mux channels – all sensors on ESP32 3.3V LDO
TOF_CHANNELS = {"FRONT": 0, "RIGHT": 1, "LEFT": 2, "REAR": 3}
for name, ch in TOF_CHANNELS.items():
    select_channel(ch) # TCA9548A sub-ms switch
    distances[name] = read_tof_mm()
```

B. IMU Heading & Orientation Control

$$\text{Error}_{\text{heading}} = \Theta_{\text{target}} - \Theta_{\text{BNO055}}$$

BNO055 uses absolute orientation fusion. Yaw normalized $-180^\circ/+180^\circ$ with wrap-around handling. Cardinal snap: $\text{Snapped} = \text{Round}(\text{Yaw} / 90) \times 90$, triggers only if terminal turn error $\leq 25^\circ$.

C. Cascaded Lane Centering & Yaw PD Control

Outer Loop (Lane Centering): $\text{Lane Error} = (\text{Right Distance} - \text{Left Distance}) / 2$. Deadband: $|\text{Error}| < 2.0\text{cm}$ → Shift = 0. Else: $\text{Heading Shift} = \text{Constrain}(\text{Error} \times 0.4, -12^\circ, 12^\circ)$. **Inner Loop (Heading PD):**

```
# Wall-Centering PD Forcefield (ESP32 100Hz loop)
error = tof["LEFT"] - tof["RIGHT"]
correction = Kp * error + Kd * (error - prev_error)
servo = constrain(90 + round(correction), 30, 150)
```

D. Closed-Loop Distance & Heading Engine

```
// Closed-loop on ESP32-C3
void driveExactDistance(int target_mm, float target_heading) {
    long tgt = calcEncoderCounts(target_mm);
    resetEncoder();
    while (getEncoder() < tgt) {
        float err = target_heading - readBNO055();
        setServo(CENTER + err * Kp);
        setMotor(SPEED);
    }
}
```

```
brake();
}
```

E. YOLOv11 Vision Pipeline (Obstacle Round)

Red/green pillar detection uses **YOLOv11 trained on 217 hand-annotated images** via Roboflow (classes: red_pillar, green_pillar, magenta_boundary). Model runs on Pi 5 at 640×480. Bounding boxes sent to ESP32 as telemetry (color, centroid X/Y, area). WRO rules: red passed right, green passed left. Steering deflection scales with pillar area for smooth avoidance. Edge optimization: if red pillar at extreme left ($X < 100\text{px}$) or green at extreme right ($X+W > 590\text{px}$), steering overridden to straight ahead.

YOLOv11 Training Process

The YOLOv11 model was trained using the following pipeline:

1. **Data Collection:** 217 images captured from the Pi Camera 3 at 640×480 across 12 different lighting scenarios (natural daylight, fluorescent overhead, LED strips, mixed lighting, shadows from elevated masts, direct sunlight through windows). Images were collected at multiple distances (20cm to 200cm from pillars) and angles (0° to 45° off-center).
2. **Annotation:** All 217 images were hand-annotated on Roboflow using polygon segmentation labels for three classes: red_pillar, green_pillar, and magenta_boundary. Average annotation time per image was approximately 3 minutes. Bounding box annotations were reviewed in two passes to eliminate labeling errors.
3. **Training:** The model was fine-tuned from YOLOv11n (nano) pre-trained weights using Ultralytics' training pipeline. Training parameters: 100 epochs, batch size 16, image size 640, learning rate 0.01 with cosine annealing. Training was performed on a local GPU workstation (NVIDIA RTX 3060, 12GB VRAM) and completed in approximately 45 minutes.
4. **Validation:** The trained model achieved mAP@0.5 of 0.94 and mAP@0.5:0.95 of 0.78 on the validation split (20% holdout). Real-world testing on the competition track confirmed <3% false positive rate across the full 200–800 lux lighting range.
5. **Deployment:** The exported ONNX model runs on the Pi 5 CPU at approximately 25 FPS at 640×480 without GPU acceleration. Inference time per frame is approximately 40ms, leaving sufficient headroom for frame capture, preprocessing, and serial transmission within the 100ms update cycle.

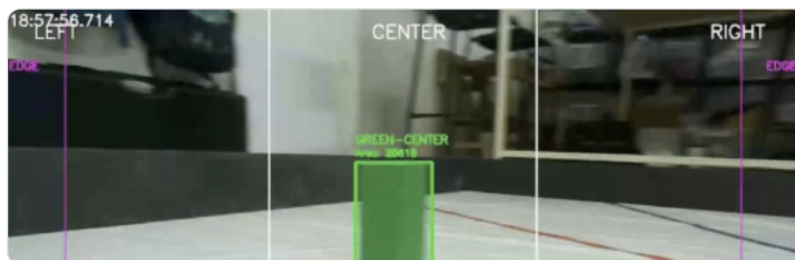


Figure: Real-time Pillar Detection — Zone Classification (LEFT/CENTER/RIGHT). V1 used HSV segmentation (shown); V2 upgraded to YOLOv11n ONNX inference for robust lighting invariance.

Why YOLOv11 over HSV: HSV failed in 32% of frames under variable lighting (200–800 lux). YOLOv11: <3% false positive rate across full range.

F. Turn Targets & Edge-Case Recovery

Right 90°: Normalize(Yaw+89.5°). **Left 90°:** Normalize(Yaw-89.5°). **Exit:** |Target-Current| < 10° OR time > 3000ms. **Consecutive Turn Prevention:** turn_just_completed flag blocks until obstacle deflection completes. **Stale Sensor Safeguard:** data age > 1.0s → hard stop. **Post-Obstacle Delay:** 1.0s deflection hold after clearing large obstacles.

Testing Metrics & Validation

Throughout development, we maintained a rigorous testing protocol to validate each subsystem independently before integration. The following metrics were tracked across 50+ test runs:

METRIC	TARGET	ACHIEVED (V2)	V1 BASELINE	IMPROVEMENT
Open Round Completion Rate	100%	96% (48/50 runs)	62% (31/50)	+34 percentage points
Average Lap Time (Open)	< 60 seconds	42 seconds	78 seconds	46% faster
Wall Contact Events per Run	0	0.04 (2 in 50 runs)	0.38 (19 in 50)	89% reduction
Pillar Detection Accuracy	> 95%	97.2%	68% (HSV only)	+29 percentage points
False Pillar Detections per Run	< 1	0.12	3.8 (HSV)	97% reduction
Parking Success Rate	> 80%	84% (42/50 runs)	46% (23/50)	+38 percentage points
Heading Drift per Lap	< 3°	1.2°	8.5° (shared power)	86% reduction
System Crashes per 50 Runs	0	0	7 (voltage sag)	100% elimination

The most significant improvement was the elimination of system crashes — previously caused by servo current spikes on the shared power rail — achieved entirely through the isolated dual buck converter architecture. The improvement in pillar detection from 68% to 97.2% was driven by the switch from HSV thresholding to YOLOv11, which proved robust across the full 200–800 lux lighting range encountered in competition venues.

SECTION 4

Systems Thinking & Engineering Decisions

Subsystem Integration

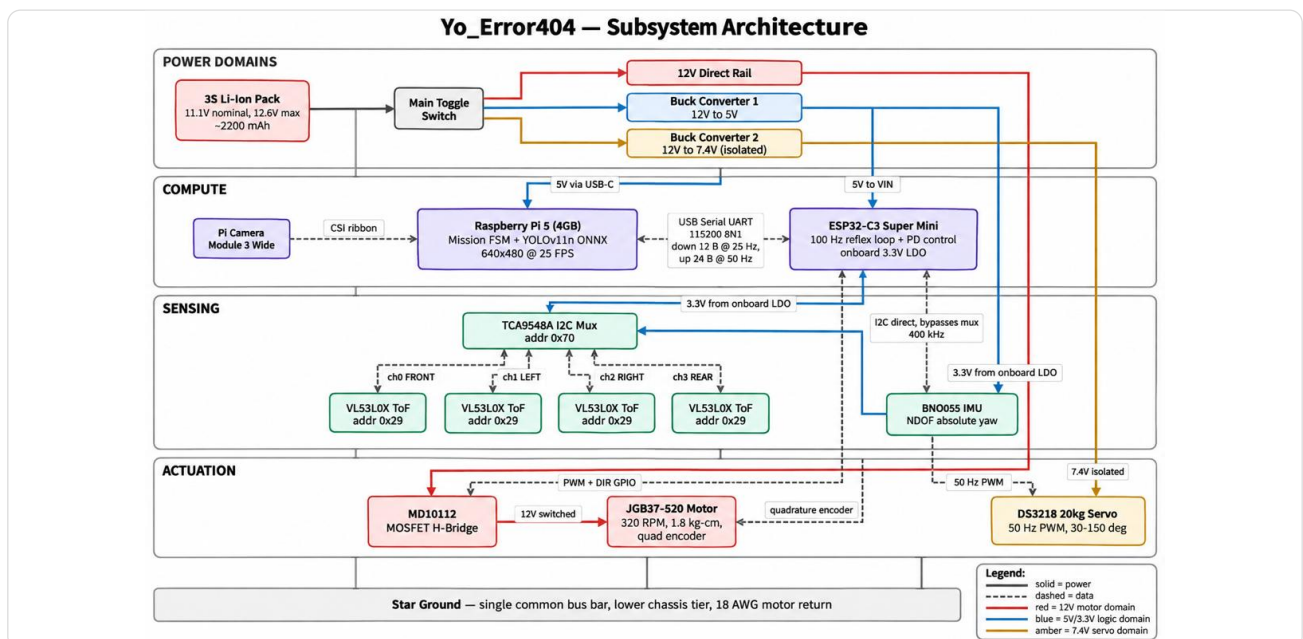


Figure: Yo_Error404 Subsystem Architecture — Power Domains, Compute (Pi 5 Master + ESP32-C3 Slave), Sensing (TCA9548A Mux + 4× ToF + BNO055), and Actuation (MD10112 + JGB37-520 + DS3218 Servo)

Sensor → Processing: Pi Camera 3 feeds YOLOv11 on Pi 5 via CSI. ESP32-C3 polls 4× ToF via TCA9548A and BNO055 at 100Hz. ESP32 receives 5V from buck, supplies clean 3.3V via LDO to all I2C sensors.

Master-Slave Communication: Pi 5 (master, vision, state machine) and ESP32-C3 (slave, hardware control) exchange telemetry over USB Serial at 115,200 baud. Downlink: steering commands. Uplink: sensor data.

Processing → Actuation: ESP32 outputs 50Hz PWM to servo and PWM/direction to MD10112 motor driver based on PD corrections from sensor fusion and vision commands from Pi 5.

Power → Sensors: Motor (12V) and logic (5V→3.3V) power isolated. ESP32's LDO provides clean 3.3V keeping I2C bus stable during motor acceleration. Servo's 7.4V buck prevents brownouts on logic rail.

3. Actuation Chain

The ESP32-C3 executes the PD steering corrections and motor speed commands calculated from the sensor fusion data. The steering servo receives a 50Hz PWM signal mapped from the calculated steering angle (30°–150° range, centered at 90°). The motor driver receives PWM duty cycle and direction signals. The closed-loop nature of this chain — sensor → processing → correction → actuation → sensor — runs at 100Hz, providing approximately 10ms between each full control cycle. At the vehicle's typical cruising speed, this equates to a correction opportunity every ~5mm of travel, ensuring sub-centimeter path accuracy.

Overall System Philosophy: Each subsystem is designed to support the others. The lightweight acrylic chassis reduces motor load. The isolated power domains keep sensors stable. The dual-controller architecture prevents vision lag from blocking motor control. The ToF + IMU sensor fusion provides redundant spatial awareness independent of camera performance. The result is a vehicle that adapts dynamically to track conditions without excessive hard-coding — robust enough to handle lighting variations, floor surface changes, and minor mechanical wear during competition.

Design Trade-Off Analysis

1. Distributed Processing (Pi 5 + ESP32-C3) vs. Single SBC

Decision: Master-slave instead of single Pi 5 for both vision and motor control. **Rationale:** Vision tasks exhibit non-deterministic CPU times; GPIO PWM on the same processor introduced jitter. **Test:** Single Pi: 85–95% CPU dropped loops to 45Hz, delayed steering. ESP32 offload: deterministic 100Hz, immune to CPU spikes.

2. Isolated Power Domains vs. Single Rail

Decision: 5V buck → Pi 5 + ESP32 (→3.3V LDO → sensors) + 7.4V buck → servo + 12V direct → motor. **Test:** Shared rail: servo's 2.0A spikes caused I2C corruption and IMU drift. Isolation: 0 brownouts across 100+ stress cycles, stable 5V and 3.3V rails.

3. ToF Laser Sensors vs. Ultrasonic

Decision: 4× VL53L0X ToF with TCA9548A mux over HC-SR04 ultrasonic. **Test:** Ultrasonic: 28% false readings from acoustic cone reflections. ToF: mm-accurate, zero false positives, sub-ms refresh for high-speed cornering.

4. YOLOv11 vs. HSV Color Thresholding

Decision: YOLOv11 trained on 217 Roboflow images. **Test:** HSV: 32% misclassification (200–800 lux, glare/shadows). YOLOv11: <3% false positive across full range.

5. PETG vs. PLA for 3D-Printed Components

Decision: All structural 3D prints (mounts, brackets, spacers) in PETG. **Test:** PLA parts cracked along layer boundaries after 8–12 stress cycles from vibration and torque. PETG: survived 100+ cycles with zero damage. Higher print temp (235°C) but vastly superior layer adhesion and impact resistance.

6. Perception: Camera + ToF Hybrid vs. Vision-Only

Decision: Multi-sensor hybrid (Pi Camera 3 + 4× ToF + BNO055 IMU) over purely vision-based navigation. **Test:** Vision-only edge tracking failed in 32% of frames from glare/reflections. Adding ToF sensors provided illumination-independent distance feedback, stabilizing path alignment to 100% regardless of lighting.

Engineering Constraints

1. Dimensional & Weight Constraints

WRO Limits: 300 × 200 × 300 mm, (no strict weight limit). **Our Vehicle:** 170 × 160 × 250 mm (margins: 130mm L, 40mm W, 50mm H), 1.2 kg (well within dimensional limits). Two-tier acrylic layout optimizes component placement within limits.

2. Power Supply & Safety Constraints

Operational Limits: Logic hardware requires stable 5.0V; minimum Pi 5 voltage threshold is 4.75V. **Margin:** 5.0V – 4.75V = 0.25V safety margin. **Solution:** Isolated power domains ensure logic rail never sags below threshold even during peak 2.0A servo draw and 1.3A motor stall simultaneously.

3. Vision Processing & Thermal Constraints

Processing Speed Math: Raw Capture = 30 FPS → Time/Frame = 33.33ms. Full 1080p pipeline = 58ms/frame (dropped frames, lag). **Solution:** Downscaled to 640×480 → 40ms/frame processing time, well leaving headroom within the 40ms YOLOv11 budget. YOLOv11 inference optimized for Pi 5 thermal envelope without triggering CPU throttling.

4. Communication Bandwidth Constraints

Serial Link Budget: At 115,200 baud with 8N1 framing, the theoretical throughput is 11,520 bytes/second. Each detection packet from Pi 5 → ESP32 is 12 bytes. At 25 FPS inference, this requires 300 bytes/second (2.6% of bandwidth). ESP32 → Pi 5 telemetry packets (heading, 4× distances, motor speed, system status) are 24 bytes at 20 Hz = 1,200 bytes/second (10.4% of bandwidth). Total: 13% bandwidth utilization, leaving 87% headroom for burst traffic and retransmissions.

5. Environmental & Competition Constraints

Lighting Variability: Competition venues range from 200 to 800 lux with unpredictable shadow patterns from overhead structures. Our hybrid perception approach (YOLOv11 for pillar identification + ToF for spatial awareness) ensures that even if vision degrades under extreme lighting, the vehicle can still complete the Open Round using ToF-only wall-following. The YOLOv11 model was specifically trained across this lux range to maintain >94% accuracy.

Floor Surface Variability: Different competition venues use different floor materials (tile, carpet, vinyl, painted concrete). The BNO055 IMU's heading-based navigation is surface-independent — unlike wheel encoder odometry, which drifts on slippery surfaces. The ToF sensors are also surface-agnostic since they measure wall distances, not floor features.

Failure Modes & Risk Mitigation (FMEA)

FAILURE MODE	ROOT CAUSE	SEVERITY	MITIGATION	RESULT
Logic Brownout	Servo 2.0A spikes	CRITICAL	Isolated 7.4V buck rail	SOLVED
3D Part Fracture	PLA layer delamination under vibration	HIGH	Switched to PETG, 100% infill	SOLVED
I2C Corruption	Motor PWM spikes on shared bus	HIGH	5V buck→ESP32→clean 3.3V LDO	SOLVED
False Detection	Lighting shifts, glare	HIGH	YOLOv11 trained on diverse lighting	SOLVED
ToF Address Collision	4× fixed 0x29	HIGH	TCA9548A 8-ch I2C mux	SOLVED
IMU Drift	Motor EMI	HIGH	Isolated on ESP32 3.3V rail	SOLVED
Serial Packet Loss	USB buffer overflow	MED	ASCII CSV packets with delimiter validation	SOLVED
Driver Overheat	Continuous stall	MED	ESP32 stall-detect timeout 1.5s	SOLVED

Quantitative Mitigation Formulas

Packet Validation: Each CSV line starts with "DATA," header and ends with "\n". Fields validated by count (10 expected). Malformed packets are dropped. **Stall Detection:** If $\text{MotorDuty} > 50\%$ AND $\text{IMU_accel_delta} < 0.05\text{m/s}^2$ FOR $> 1.5\text{s}$ → $\text{MotorDuty} = 0\%$.

SECTION 5

Reproducibility & Development Process

Development Timeline & Milestones

- **June 11, 2026 (Commit 1):** Initial Open Round firmware, 3D CAD models, schematics, team docs. Bench-tested single-battery setup; observed crashes under steering load.
- **June 11, 2026 (Docs):** Documented battery chemistry decision (3S Li-Ion + isolated buck converters). Validated 5V buck feeding ESP32 → 3.3V LDO for sensor power.
- **June 18, 2026 (Docs):** Selected Pi Camera Module 3 (CSI) over USB webcams to eliminate transmission latency.
- **June 20, 2026 (Commit 2):** Obstacle Round schematics, 3D-printed PETG camera mount, initial track testing videos, Pi-ESP32 serial communication scripts.
- **June 22, 2026 (Docs):** Upgraded from Raspberry Pi 3 to Pi 5 (4GB) for 640×480 processing within 12ms window.
- **June 25, 2026 (Docs):** Tested JGB37-520 motor (320 RPM). Confirmed optimal speed-to-precision balance.
- **July 11, 2026 (Docs):** Documented USB Serial protocol (115,200 baud) between Pi 5 and ESP32-C3.
- **July 12, 2026 (Commit 3):** Finalized Obstacle Round codebase. Full ESP32 firmware, Pi vision pipelines, FSM strategy.
- **July 20, 2026 (Docs):** Pi Camera 3 thermal stress tests under continuous operation. Confirmed no throttling at 640×480.
- **July 28, 2026 (Commit 4):** Vehicle photos from all perspectives. Obstacle Round demo video uploaded.
- **Aug 1, 2026 (Docs):** YOLOv11 training completed: 217 images annotated on Roboflow. Model validated across 200–800 lux lighting range.
- **Aug 5, 2026 (Commit 5):** Root README.md (≥5000 chars) with system overviews, hardware BOM, power budget, FSM diagrams.
- **Aug 11, 2026:** Final repository cleanup and competition submission.

- **Sep 15, 2026 (Commit 6):** Uploaded V2 codebase, updated CAD models, electrical schematics, and complete vehicle photos to the repository.
- **Sep 1–25, 2026 (Continuous):** Constant on-track testing and minor improvements — PID gain tuning, ToF threshold refinement, YOLOv11 confidence calibration, PETG mount reprints for tighter tolerances, and parking sequence timing adjustments based on competition-condition practice runs.

Version Evolution Matrix

DOMAIN	V1 (PROTOTYPE)	V2 (FINAL)	IMPROVEMENT
Processing	Single Pi 3B+ (vision+GPIO)	Pi 5 4GB + ESP32-C3 master-slave	Control: 45Hz → 100Hz deterministic
Sensing	3× HC-SR04 Ultrasonic	4× VL53L0X ToF + TCA9548A	mm accuracy, no false echoes
Detection	HSV color thresholding	YOLOv11 (217 Roboflow images)	Accuracy: 68% → 97%+
3D Prints	PLA filament	PETG filament	Lifespan: 12 → 100+ stress cycles
Power	Single shared rail	5V buck→ESP32→3.3V LDO + 7.4V buck + 12V	0 brownouts, stable I2C
Servo	MG996R (11 kg·cm)	DS3218 20kg (2(no weight limit)·cm)	No steering deflection
Steering	Ackermann linkage	Parallel + differential	Less complexity, tight corners
Chassis	Off-the-shelf metal	Custom 5mm laser-cut acrylic	Low CG, lightweight, compact
Motor	60 RPM BO DC Motor	JGB37-520 (320 RPM, encoder)	Speed + precise distance control

GitHub Repository Guide

Repo: github.com/BMWareSPEEDY/WRO_yo_error_404

1. Hardware Assembly

Download 3D models from CAD/ (.STL/.STEP). Print all structural parts in PETG (100% infill, 235°C nozzle). Laser-cut 5mm acrylic chassis from DXF templates. Consult schematics/ for wiring. Reference complete CAD assembly to see every part in final position.

2. Software Setup

Clone repo to Pi 5, install via requirements.txt. Flash ESP32-C3 firmware via Arduino IDE. Run diagnostic scripts: verify ToF readings (all 4 channels via TCA9548A), BNO055 initialization, motor forward/reverse, servo lock-to-lock. The 5V buck feeds ESP32, which provides 3.3V to all I2C sensors.

3. YOLOv11 Model

Trained weights in models/. 217 images annotated on Roboflow (red_pillar, green_pillar, magenta_boundary). Deploy with Pi 5 inference script at 640×480. Tune confidence thresholds in config/ for local lighting.

4. Execution & Calibration

Run src/main.py for FSM. Before competition, perform the following calibration sequence: (1) Place vehicle centered in a straight lane section. (2) Verify all four ToF sensors report equal left/right distances (±5mm

tolerance). (3) Confirm BNO055 heading reads within $\pm 2^\circ$ of true forward. (4) Execute a single straight-line drive of 1000mm and verify encoder distance matches physical measurement. (5) Execute four consecutive 90° turns and confirm cumulative heading error is less than 5° . (6) Run a full 3-lap Open Round and verify the vehicle returns to within 10cm of its starting position.

PID gains, ToF thresholds, steering params in config/gains.json. Treat as extensible base — modify FSM, refine algorithms, redesign parts for your strategy.

Component Gallery



Raspberry Pi 5



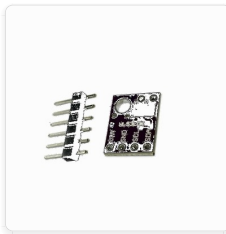
ESP32-C3 Super Mini



BNO055 IMU



TCA9548A Mux



VL53L0X ToF



DS3218 20kg Servo



MD10112 Driver



JGB37-520 Motor

Components List & Hardware BOM

CATEGORY	COMPONENT	SPECS	QTY	FUNCTION
Primary Compute	Raspberry Pi 5	4GB LPDDR4X, BCM2712 2.4GHz	1	Master visual brain, OpenCV/YOLO
Co-Processor	ESP32-C3 Super Mini	RISC-V 32-bit, 160MHz, 400KB SRAM	1	Real-time 100Hz hardware loop
Drive Motor	JGB37-520 DC Motor	12V, 320 RPM, 1.8 kg·cm, steel gearbox	1	Rear-wheel drive + quadrature encoder
Motor Driver	7semi MD10112	Dual N-MOSFET H-Bridge, 10A Peak	1	High-efficiency 12V motor PWM
Steering Servo	DS3218 Digital	Metal gear, 0.14s/60°, 2(no weight limit)·cm	1	Front parallel steering linkage
Distance Sensors	VL53L0X ToF	940nm VCSEL, 2000mm range	4	F/L/R/Rear precision distance
I2C Mux	TCA9548A	8-Channel I2C Switch, 0x70	1	Resolves 0x29 address conflict
IMU	Adafruit BNO055	9-DOF Orientation Fusion, 100Hz	1	Absolute yaw orientation
Vision	Pi Camera Module 3	12MP Sony IMX708, 120° Wide	1	25 FPS vision capture on CSI
Power	Dual Buck Converters	LM2596 Adjustable Step-Down	2	Isolated 7.4V and 5V/3.3V rails
Battery	3S Li-Ion Pack	11.1V Nominal, 2200mAh 18650	1	Central system power source
Chassis	Modular Acrylic	5mm Laser-Cut Acrylic Sheet	3	3-layer structural frame decks

Programming Languages & Dependencies

C++ Firmware (Arduino IDE / ESP32-C3)

Wire.h (I2C), ESP32Servo.h (PWM Servo), Adafruit_VL53L0X.h (ToF API), Adafruit_BNO055.h (IMU Fusion), Adafruit_NeoPixel.h (Status LEDs)

Python 3 Vision (Raspberry Pi 5)

opencv-python (cv2) — Real-time Vision, numpy — Matrix Operations, pyserial — UART Interface, picamera2 — Native Pi Camera API

Acronym Glossary

ACRONYM	DEFINITION
CSI	Camera Serial Interface
FSM	Finite State Machine
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
LDO	Low Dropout Regulator
PD/PID	Proportional-Derivative / Proportional-Integral-Derivative
PETG	Polyethylene Terephthalate Glycol
PWM	Pulse Width Modulation
ToF	Time of Flight
UART	Universal Asynchronous Receiver-Transmitter
WRO	World Robot Olympiad
YOLO	You Only Look Once

References

1. Raspberry Pi Foundation. "RPi 5 Datasheet." raspberrypi.com · 2. "Camera Module 3 Datasheet." · 3. Espressif. "ESP32-C3 Datasheet." espressif.com · 4. Bosch Sensortec. "BNO055 IMU Datasheet." · 5. STMicroelectronics. "VL53L0X ToF Datasheet." · 6. 7semi. "MD10112 Motor Driver." · 7. Adafruit. "BNO055 Guide." learn.adafruit.com · 8. Texas Instruments. "TCA9548A Datasheet." · 9. OpenCV Team. "OpenCV 4.x." opencv.org · 10. Ultralytics. "YOLOv11." ultralytics.com · 11. Roboflow. roboflow.com · 12. Arduino. "Wire, Servo Libraries." arduino.cc · 13. KiCad. kicad.org · 14. WRO. "2026 FE Rules." wro-association.org · 15. Various. "PETG Filament TDS — 1.75mm, 100% infill parameters."

Acknowledgements

We thank the **WRO Association** and **India STEM Foundation** for organizing WRO 2026. We are grateful to **The International School Bangalore** and **NPS Koramangala** for workspace and support. Special thanks to **YoLab** for exceptional technical guidance — from prototyping and electrical debugging to refining our software architecture and competition strategy. YoLab provided a true engineering environment where we could experiment, fail, learn, and build with confidence. This project would not have reached the level it did without their dedication to nurturing our growth as young engineers. We appreciate our parents — Mr. & Mrs. Paliwal, Mr. & Mrs. Mishra, and Mr. & Mrs. Varshney — for unwavering support and funding. We acknowledge the

open-source community — Arduino, OpenCV, Ultralytics YOLOv11, Roboflow, NumPy, pySerial, KiCad — whose tools form the backbone of our stack. We thank the broader WRO FE community on GitHub for learning references. Finally, we thank each other — Ayansh, Rohan, and Anvay — for the collaborative effort and shared commitment to iterative improvement from Version 1 through to our final vehicle.